AI Fundamentals

Contents

1. AI vs ML vs Deep Learning 2
2. Supervised, Unsupervised and Reinforcement Machine Learning Approach 4
3. Neural Networks 10
4. Tokens 12
5. Embeddings 15
6. Transformers 18
7. Large Language Models 21
8. LLM Inference 24
9. Foundational Models 31
10. Multimodal AI 34
11. Open vs Closed AI Models 37
12. AI Hallucinations 40

1. AI vs ML vs Deep Learning

Understanding how Artificial Intelligence, Machine Learning and Deep Learning relate

1.1 Explanation

Think of Artificial Intelligence (AI) as the big goal: getting computers to do things that normally need human intelligence, such as understanding language, recognizing faces or making decisions. Machine Learning (ML) is one way to reach that goal: instead of writing every rule by hand, you show the computer lots of examples and let it work out the rules itself. Deep Learning (DL) is a particular kind of machine learning that uses large neural networks with many layers, loosely inspired by how neurons in the brain connect.

1.2 Key terms

Term	What it means
1. Artificial Intelligence (AI)	The broad field of building systems that perform tasks requiring human-like intelligence, whether through rules, search or learning.
2. Machine Learning (ML)	A subset of AI in which systems learn patterns from data rather than following explicitly programmed rules.
3. Deep Learning (DL)	A subset of ML that uses neural networks with many layers to learn representations directly from raw data.
4. Neural network	A model made of layers of connected artificial neurons, each computing a weighted sum followed by a non-linear function.
5. Training data	The examples a model learns from. The quality and quantity of this data largely decide how well the model performs.
6. Model	The result of training: a learned set of parameters that takes new input and produces a prediction or output.
7. Generative AI	Deep learning models that create new content such as text, images, code or audio, rather than only classifying or predicting.

1.3 Architecture diagram

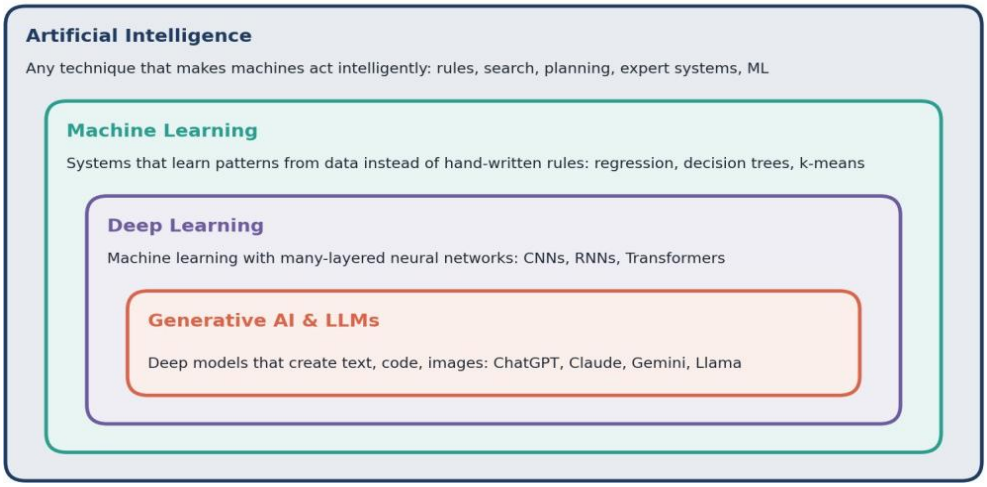


Figure 1: How AI, ML, Deep Learning and Generative AI nest inside each other

1.4 Comparison table

Feature	Artificial Intelligence (AI)	Machine Learning (ML)	Deep Learning (DL)
Definition	Broad field of making machines perform tasks requiring intelligence	A subset of AI where machines learn patterns from data	A subset of ML using multi-layer neural networks
Main idea	Make machines act intelligently	Make machines learn from data	Make machines learn complex patterns automatically
How it works	Rules, logic, search, learning, or a combination	Algorithms learn from examples and data	Neural networks learn representations from large amounts of data
Human involvement	Can require manually defined rules	Humans often select features and prepare data	Requires less manual feature engineering
Computing power	Low to high, depending on approach	Moderate	Usually high
Examples	Chatbots, game AI, expert systems, recommendation systems	Spam detection, fraud detection, credit scoring	Face recognition, speech recognition, autonomous driving
Typical algorithms	Rule-based systems, search, planning, ML	Decision trees, linear regression, random forests, gradient boosting	CNNs, RNNs, transformers, deep neural networks
Relationship	Biggest category	Subset of AI	Subset of ML

1.5 Real-world examples

Example	AI	Machine Learning (ML)	Deep Learning (DL)
1. Face Unlock on a phone	Phone identifies whether the person should be allowed to unlock	Learns patterns from examples of faces	Uses deep neural networks to recognise complex facial features
2. YouTube/Netflix recommendations	System recommends content you may like	Learns from your watching history, likes and searches	Deep models can learn complex relationships between users, videos and viewing behaviour
3. Self-driving cars	Car makes decisions such as when to stop, turn or change lanes	Learns patterns from driving data	Deep learning identifies pedestrians, vehicles, traffic signs and road conditions from cameras/sensors
4. Voice assistants	Assistant understands requests and performs tasks	Learns patterns in speech and language	Deep neural networks convert speech to text and understand language

Example	AI	Machine Learning (ML)	Deep Learning (DL)
5. Email spam detection	System decides whether an email is spam	Learns from examples of spam and legitimate emails	Deep learning can analyse complex patterns in email text and behaviour

1.6 Summary: How AI, ML and Deep Learning are connected

- **Artificial Intelligence (AI):** The broad field of building systems that perform tasks requiring human-like intelligence, such as understanding language, recognizing images, and making decisions. AI can use predefined rules or learn from data.
- **Machine Learning (ML):** A branch of AI where models learn patterns from training data to make predictions on new, unseen data. Examples include decision trees, linear regression, and random forests.
- **Deep Learning (DL):** A branch of ML that uses neural networks with multiple layers to learn complex patterns automatically. It powers applications such as image recognition, speech processing, and modern chatbots.

2. Supervised, Unsupervised and Reinforcement Machine Learning Approach

2.1 Supervised Learning

Supervised learning is learning with an answer key. You give the model thousands of examples where the correct answer (the label) is already known, such as emails marked “spam” or “not spam”, and it learns to predict the label for new examples

Types of supervised learning

- Classification predicts a category. Examples: spam or not spam, fraud or legitimate, which digit (0 to 9) is in an image.
- Regression predicts a continuous number. Examples: house price, delivery time, next month’s sales.

Workflow

1. **Collect labelled data: inputs with their known correct outputs.**
2. **Prepare features: convert inputs into numbers the model can use and clean missing values.**
3. **Split the data: typically, 80% for training and 20% for testing.**
4. **Train: the model predicts, measures its error against the labels (the loss), and adjusts its parameters to reduce that error.**
5. **Evaluate: measure accuracy or error on the test data the model has never seen.**
6. **Predict: use the trained model on new, unlabelled inputs.**

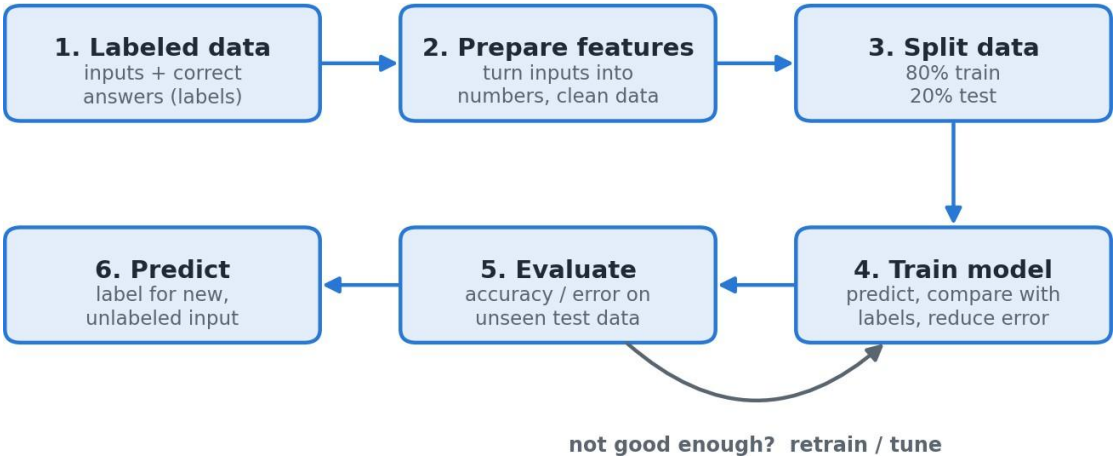


Figure 2: Supervised learning workflow

Common algorithms

Algorithm	Type	How it works
Linear Regression	Regression	Fits a straight line (or plane) between features and a numeric output.
Logistic Regression	Classification	Estimates the probability of a class, usually yes/no.
k-Nearest Neighbors (k-NN)	Both	Predicts from the labels of the k most similar training examples.
Decision Tree	Both	Learns a set of if/else rules that split the data.
Random Forest	Both	Combines many decision trees and takes a vote or average.

Real-world use cases

- **Email spam and phishing filters**
- **Credit card fraud detection**
- **Medical image diagnosis (e.g. X-ray screening)**
- **Handwriting and postcode recognition**
- **House price and sales forecasting**

Limitations of Supervised Learning

- Performance depends heavily on the quality and accuracy of the labels.
- May perform poorly when the new data is very different from the training data.
- Requires a large amount of labelled data, which can be expensive and time-consuming to create.

2.2 Unsupervised Learning

Unsupervised learning is learning without an answer key. The model gets raw data and has to find structure on its own, for example grouping customers with similar buying habits, even though nobody told it what the groups should be.

Types of unsupervised learning

- Clustering groups similar data points together (e.g. customer segments).
- Anomaly detection finds data points that do not fit the normal pattern.
- Association rules find items that frequently occur together.

Workflow

1. Collect raw data: no labels are required.
2. Select and scale features so no feature dominates because of its units.
3. Run the algorithm to find groups, patterns or outliers.
4. Interpret the result: the algorithm returns group numbers; a person inspects and names each group.
5. Act on it: targeted marketing, investigation of anomalies, or input to other models.

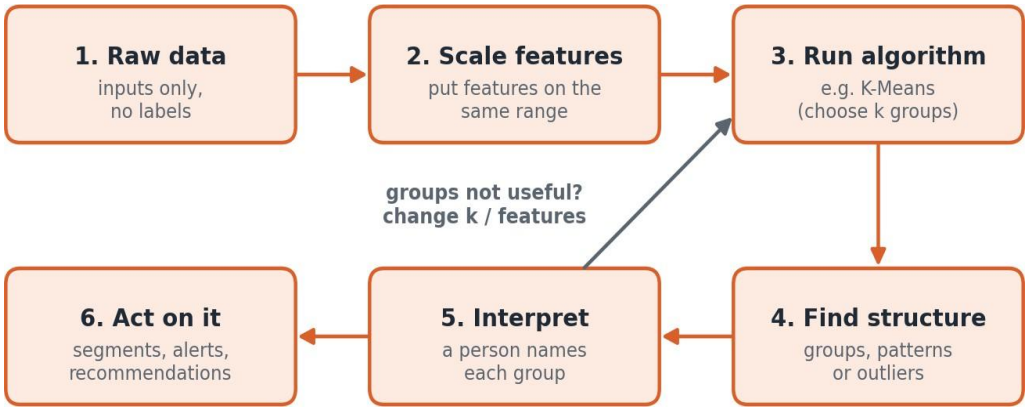


Figure 3: Unsupervised learning workflow

Common algorithms

Algorithm	Type	How it works
K-Means	Clustering	Splits data into k groups around centre points.
Hierarchical Clustering	Clustering	Builds a tree of groups within groups.
DBSCAN	Clustering	Finds dense groups of any shape and marks noise points.
Isolation Forest	Anomaly detection	Flags points that are easy to isolate from the rest.
Apriori	Association rules	Finds item combinations that often appear together.

Real-world use cases

- **Customer segmentation for marketing**
- **Fraud and network intrusion detection (anomalies)**
- **Product and content recommendations ("people also viewed")**
- **Grouping research articles into topics**
- **Detecting clusters of suspiciously similar manuscripts (paper mills)**

Limitations of Unsupervised Learning

- **Results can be difficult to interpret or validate, because there is no predefined correct answer.**
- **The quality of the results can depend strongly on feature selection and scaling.**
- **Some algorithms require assumptions about the shape, size, or density of clusters.**
- **The number of clusters may need to be chosen in advance for methods such as K-Means.**
- **Detected patterns or anomalies may not always have a useful real-world meaning.**

2.3 Reinforcement Learning

Reinforcement learning (RL) is learning by trial and error, like training a dog with treats. An agent takes actions in an environment, receives a reward (or penalty), and gradually learns which actions lead to the most reward over time.

Rewards and penalties

- **Rewards encourage desired behaviour, e.g. +10 for reaching the goal.**
- **Penalties discourage undesired behaviour, e.g. -10 for falling into a hole.**
- **Small step penalties (e.g. -1 per move) push the agent toward the shortest route.**
- **The agent optimizes cumulative reward, so it can accept a small penalty now for a larger reward later.**

Workflow

1. **The agent observes its current state.**
2. **It chooses an action: sometimes random (exploration), usually the best-known action (exploitation).**
3. **The environment returns the next state and a reward or penalty.**
4. **The agent updates its estimate of how good that action was**
5. **Repeat over many episodes until the policy stops improving.**

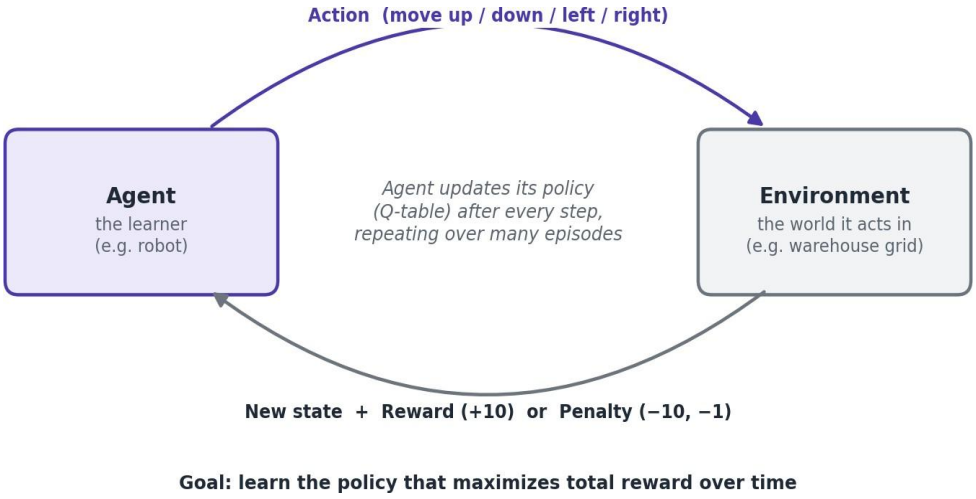


Figure 4: Reinforcement learning loop between agent and environment

Common algorithms

Algorithm	How it works
Q-Learning	Learns a table of values for every state and action.
Deep Q-Network (DQN)	Replaces the table with a neural network for very large state spaces (e.g. game screens).
Policy Gradient / PPO	Learns the policy directly; used in robotics and to fine-tune chat models (RLHF).
Multi-Armed Bandits	Simplest form: repeatedly choose between options to find the best-paying one.

Real-world use cases

- Game-playing AI (AlphaGo, Atari games)
- Robotics and warehouse automation
- Self-driving car decision-making
- Traffic-signal control
- Fine-tuning AI assistants with human feedback (RLHF)
- Recommendation systems that learn from clicks over time

Limitations of Reinforcement Learning

- Training can be slow because the agent may need many episodes to learn a good policy.
- Designing suitable rewards and penalties can be difficult and can strongly affect what the agent learns.

2.4 Practical Demo

Check out this example to understand how these approaches work: Machine Learning Approaches.

2.5 Key Terminology

Term	Meaning	Applies to
Training data	The examples a model learns from.	All
Features	Measurable inputs the model uses (e.g. pixel values, income).	Supervised, Unsupervised
Labels	The correct answer attached to each training example.	Supervised
Model	The learned function that maps inputs to outputs.	All
Training	Adjusting the model's parameters so it performs better.	All
Test data	Data held back from training to check performance on unseen examples.	Supervised
Loss function	A measure of how wrong the model's predictions are.	Supervised
Overfitting	When a model memorises training data and performs poorly on new data.	Supervised
Prediction (inference)	Using a trained model on new input.	All
Cluster	A group of similar data points found by the algorithm.	Unsupervised
Centroid	The centre point (average) of a cluster.	Unsupervised
Agent	The learner and decision maker.	Reinforcement
Environment	Everything the agent interacts with; returns states and rewards.	Reinforcement
State	The agent's current situation.	Reinforcement
Actions	The choices the agent can make in a state.	Reinforcement
Rewards	Positive feedback for good outcomes.	Reinforcement
Penalties	Negative feedback (negative rewards) for bad outcomes.	Reinforcement
Policy	The agent's strategy: which action to take in each state.	Reinforcement
Episode	One complete attempt from start to finish.	Reinforcement
Exploration vs exploitation	Trying new actions to learn versus using the best-known action.	Reinforcement

2.6 Comparison of Supervised, Unsupervised and Reinforcement Learning

Aspect	Supervised	Unsupervised	Reinforcement
Data	Labeled (input + correct output)	Unlabeled (input only)	No fixed dataset; generated by interaction
Feedback	Correct answer for every example	None	Reward or penalty after actions
Goal	Predict labels for new data	Discover hidden patterns or groups	Learn a policy that maximizes cumulative reward
Output	Category or number	Clusters, reduced features, anomaly scores	Best action for each state
Key algorithms	Linear/Logistic Regression, k-NN, Decision Tree, Random Forest, SVM	K-Means, DBSCAN, Hierarchical, PCA	Q-Learning, SARSA, DQN, PPO
Data cost	High: labeling takes effort	Low: raw data is enough	Needs a simulator or safe environment
Main challenge	Getting enough good labels; overfitting	Results can be hard to interpret or validate	Slow training; reward design is tricky
Example	Spam filter	Customer segmentation	Game-playing AI

3. Neural Networks

The building block of deep learning: many simple units that together learn complex patterns.

3.1 Explanation

A neural network is a stack of simple calculators called neurons. Each neuron takes some numbers in, multiplies each by an importance score (a weight), adds them up with a small adjustment (a bias), and passes the result through an activation function that decides how strongly it "fires".

3.2 Key terms

Term	What it means
1. Neuron	The basic unit of a network: it computes a weighted sum of its inputs plus a bias, then applies an activation function.
2. Weight	A learnable number on each connection that scales how much an input influences a neuron. Weights are the “knowledge” of the model.
3. Bias	A number that helps the neuron adjust its answer.
4. Activation function	A rule that helps the neuron decide what output to give. A non-linear function that lets networks model complex, non-linear relationships such as ReLU (Rectified Linear Unit).
5. Backpropagation	An algorithm that helps the network learn from its mistakes by changing the weights.

3.3 Architecture diagram

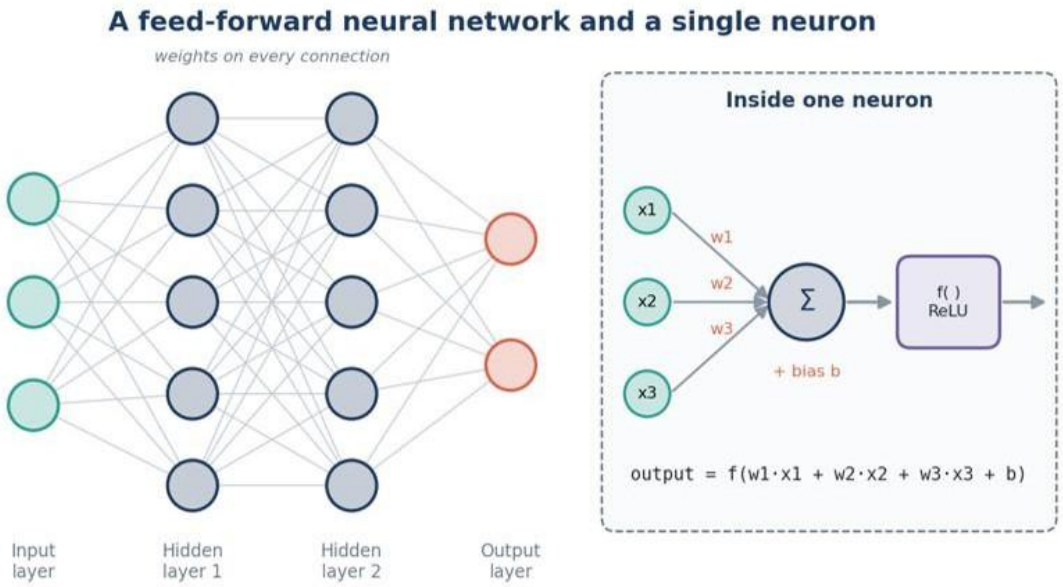


Figure 5: A feed-forward neural network and a single neuron

3.4 How it learns

A brand-new network knows nothing. All its weights start as random numbers, so its first answers are pure guesses. It learns from examples where we already know the right answer, such as thousands of photos already labelled “cat” or “dog”. Training repeats one simple loop:

1. Guess. Show the network an example. The numbers flow forward through the layers and it gives an answer, for example “70% sure this is a dog”.
2. Check. Compare the guess with the right answer. A score called the loss measures how wrong it was: confidently wrong gives a big loss, nearly right gives a small one.

3. **Find the blame. Backpropagation works backwards through the network to figure out how much each weight contributed to the mistake.**
4. **Adjust. Every weight is nudged a tiny step in the direction that would have made the answer less wrong. This step is called gradient descent.**
5. **Repeat. Do this for every example, many times over. Each full pass through all the examples is called an epoch.**

An everyday comparison: think of adjusting the hot and cold taps in a shower. You start at a random setting, feel the water, turn the taps slightly in the right direction, and repeat until it feels right. The weights are the taps, the loss is how far from the water is from the temperature you want, and nobody tells the network the right settings. It finds them by trial and correction.

Two things decide whether training works:

- **Enough good examples. The network can only learn patterns that appear in its training data, so more varied, accurate examples mean better results.**
- **Testing on unseen examples. We always hold back some examples the network never trains on. If it does well on those too, it has genuinely learned the pattern. If it only does well on the training examples, it has just memorised them. This is called overfitting, like a student who memorises past exam answers but can't handle a new question.**

3.5 Where it is used

- **Computer vision: photo tagging, OCR, medical scans, quality inspection on production lines.**
- **Speech: voice assistants, transcription, speaker identification.**
- **Language: translation, search ranking, and every modern LLM.**
- **Tabular and time series: forecasting, recommendation systems, fraud scoring at scale.**
- **Check out this real-world example which trains a neural network model: Handwriting Recognition.**

4. Tokens

- **Small piece/unit of text that LLM or AI model processes as an input or produces as an output.**
- **That unit can be:**
  - **part of word (sub-word) e.g. ing, able, ful etc.**
  - **complete word e.g. cat, dog, the, play**
  - **whitespace**
  - **number**

- special character, emoji etc.

4.1 Tokenization

- Process of converting raw text into tokens that the model can process.
- LLMs are mathematical models so it only understands numbers.
- To solve this problem, we use Tokenizer which acts as language translator that breaks text into pieces and assigns Token ID to each piece from massive, pre-built master dictionary i.e. Vocabulary/Vocab dictionary.
- Example: Imagine models vocabulary looks like:

Token (piece of text)	Token ID
trans	412
form	105
er	89
s	21
ing	34

If we input word "transformers", the tokenizer matches the biggest fragment it can find: ["trans", "form", "er", "s"] (4 tokens) and if we input "transforming" then: ["trans", "form", "ing"] (3 tokens).

- If a model has vocabulary size of 100,000 it means model only knows 100,000 specific token variations. These can be whole words, sub-words, punctuations, spaces combined with text, numbers etc. Everything we ever type into AI model must be forced into combination of those 100,000 tokens.
- Example: Vocabulary size of different models
  - Older/English-focused models (LLaMA2, GPT-3.5): ~32,000 to 50,000 tokens
  - Modern standard models (LLaMA3): ~128,000 tokens
  - Large multilingual models (OpenAI o200k_base, LLaMA4): ~200,000+ tokens
- Modern LLMs uses sub-word tokenization, which breaks a text into common patterns, prefixes, suffixes, and punctuation. This will help in balancing vocabulary size and handle rare or out-of-vocabulary words.
- Example: A sub-word tokenizer might break playing into ["play", "ing"] (2 tokens), while "hello" might remain ["hello"] (1 token). This result may vary, depending on the type of tokenizer and vocabulary.

Why LLM don't use words i.e. Word-based tokenization?

Consider words like play, playing, played, player, playground. If every variation becomes own token, then size of vocabulary becomes unnecessarily large. Also, word-based tokenizer would trouble in handling words it has never seen like SolidGoldMagicarp, the model may glitch out, panics, or start speaking non-sense. So, instead of requiring every word to exist in the vocabulary, the tokenizer can construct word from smaller known pieces i.e. ["Solid", "Gold", "Magic", "arp"].

4.2 Token ID

- Unique number i.e. numerical identifier that LLM model assign to a specific token in tokenizer's vocabulary so it can process languages mathematically.

- **As LLM is a mathematical model so it only understands numbers. So, after tokenizer generates Token IDs and then this IDs are mapped to numerical vector representations called "Embeddings".**
- **Example: Suppose we enter "AI is changing the world!", then tokenizer may produce something like ["AI", " is", " changing", " the", " world", "!"] (6 tokens – in modern LLMs a whitespace before word is merged into next token itself).**

  Then each token is assigned with unique Token ID i.e. [17527, 382, 13784, 290, 5992, 0] (This IDs and overall token count may vary, it depends on the tokenizer and model we are using).

4.3 Generic Rules of Token

- **Rule of thumb: For English text, 1 token is about 4 characters i.e. 100 tokens are nearly equal to 75 words. These are rough estimates for English text, not fixed rules.**
- **Common vs Rare words: Common words like the, and, cat, or the words which we use in day-to-day life usually count as 1 token. Rare, long, or complex words may break into multiple sub-words (it depends on tokenization method model is using), like "unbelievable" which is a long word might be split down into ["un", "believe", "able"] (3 tokens).**
- **Punctuation Count: Punctuation mark like (? "" ; ,) can be represented as its own individual token and has unique Token ID or as a part of token, depending on the tokenizer.**
- **Space Count: In the modern tokenizers like OpenAI's cl100k_base, a space before word in a sentence is merged into token itself.**
  - **The word " hello" is one token**
  - **The word "hello" is one token, but both have completely different Token ID.**
- **Case Sensitivity Matters: Capitalization changes the Token ID. "apple" and "Apple" can have different Token IDs.**

4.4 Token Limits and Why Token Count Matters

- **Context window: This is the total number of tokens the model can process and use within a single request i.e. within single conversation (including our prompt or input and model's generated response or output). If conversation exceeds this limit, the model may need to remove or exclude some earlier content from the context supplied to model. Every model has a context window size ranges from 8000 to 128,000 tokens (it again depends on or varies by model, so always check the documentation for the specific model we are using).**
- **Cost: AI companies like OpenAI, Anthropic, Google don't bill us by the word or by the hour. They usually bill us per 1 million tokens. Also, input and output token may have different prices.**
- **Language Bias: Most models are trained mostly on English, non-English languages like Hindi, Spanish etc. required a greater number of tokens.**

  Example: The "hello" can be 1 token but its Hindi translation "नमस्ते" can have 4 to 6 tokens. This makes models in non-English languages slower and more expensive.

4.5 Practical Visualization

1. **Open either OpenAI tokenizer or TikTokenizer (for this we can select a particular model we want to test our text/prompt in) in browser.**
2. **Enter prompt/sentence like "The capital of India is Delhi". Observe total number of characters, number of tokens, how the sentence is divided into individual tokens, and Token IDs.**

3. **Test different types of text like normal words, with punctuation, long words, numbers, code, different languages, emojis etc. Also, you can take one sentence and gradually modify it and observe how token count changes.**
4. **Notice the actual word count and token count.**

5. Embeddings

5.1 Vector

- **It is an ordered collection (array) of numbers used to represent something mathematically. Each number represents a value along a particular dimension.**
- **In AI, vectors are commonly used to represent data such as text, images, or audio in the form that model can process and compare.**

5.2 Embeddings

- **Vector representations of data like text, images, audio etc. used in generative AI to convert complex information into a format that model can process.**
- **Representing text, images, audio as points in a continuous vector space.**

- These vectors capture semantic relationships within data, allowing AI models to understand context and generate relevant outputs.
- E.g. [2,4] is a vector in 2-dimensional space, [3,5,7] is a vector in 3-dimensional space.
- But in LLMs, the embedding vectors aren't usually two or three dimensions, they're often containing hundreds or thousands of dimensions i.e. "N-dimensional vector space". We as a human can't visualize them but computers can still represent these vectors and perform mathematical computation with them.

5.3 Why Vector Embeddings

- With Tokens and Token IDs we solve the problem of turning text into numbers. But Token IDs are arbitrary values i.e. random addresses. If "apple" and "banana" are assigned with ID 40 and 1842, then model has zero clue about their features like that one is red and other is yellow and both can be eaten.
- Embeddings solve this problem by replacing single random ID with long string of numbers i.e. Vector (a list of co-ordinates).

5.4 Types of Vector Embeddings / Types of Data Represented with Embeddings

1. Word Embeddings

- Numeric vectors which represent individual words as vectors.
- It translates single word into vectors, and similar words are placed together.
- These can be helpful to show relationships between words, like "King" and "Queen" are related in the same way as "man" and "woman".
- Common word embedding models are Word2Vec, GloVe, FastText etc.

2. Sentence and Document Embeddings

- It represents larger pieces of text i.e. sentence by capturing the overall meaning and context of an entire sentence or document, not just individual word.
- It plays crucial role in various NLP applications, such as sentiment analysis, question answering, information retrieval etc.
- Models like Doc2Vec, USE, SBERT, and ELMO have been trained on massive amounts of pre-trained data, such as Google News and Wikipedia.

3. Image Embeddings

- It converts images into vectors and captures visual features like shapes, colours, and textures.
- They enable tasks like image recognition, classification, object detection and other visual recognition tasks.
- Image embeddings are created using deep learning models like Convolutional Neural Networks (CNNs) which includes models like VGG, ResNet, Inception and EfficientNet. These models are pre-trained on large-scale image datasets and can be used as powerful feature extractors.
- E.g. Image embedding might help model to recognize whether a given picture is a dog or cat.

4. Audio Embeddings

- It converts sounds into vectors and captures features like pitch, tone, rhythm.
- They are particularly used in speech recognition, audio classification, emotion detection, and music analysis.

- These are created using Recurrent Neural Networks (RNNs), CNNs, and hybrid models that combine both.
- Common models are Wav2Vec, Whisper Encoder.

5.5 How to Create Vector Embedding

1. **Choose an embedding model**
   - Select a pre-existing embedding model suitable for our data and task.
   - For text, we might choose Word2Vec, Glove, or BERT.
2. **Prepare data**
   - Clean and pre-process the data which is compatible with the chosen embedding model.
   - For text, this can include tokenization and possibly pre-processing. For images, we may need to resize and normalize the pixel values etc.
3. **Load or train the embedding model**
   - We can either use an already trained model, which already knows how to create vector representations, or train a new model using our prepared data so it can learn how to create those representations.
4. **Generate embeddings**
   - For each data point, use the trained or loaded model to generate embeddings.
   - For example, if using word embedding model, input a word to get its corresponding vector representation.

5.6 Vector Representation

- It is a way to represent a data as a vector (collection of numerical values), so that models can process, compare, and analyse data mathematically.
- E.g. Suppose we have a text "I love dogs." an embedding model may represent it as a vector i.e. [0.21, -0.45, 0.73, 0.18, 0.52, ...]

5.7 Semantic Similarity

- It refers to how two pieces of information like words, sentences, or phrases are related in meaning, even when they used different words or sentence structures.
- It measures how close or how different two pieces are in terms of meaning and context.
- E.g. Consider two sentences, Sentence 1: "How can I reset my password?" and Sentence 2: "I forgot my password, how can I change it?". Here the wording is different, but both sentences have a similar meaning.

  Now Embedding model converts both the sentences into vectors: Vector (Sentence 1) = [0.21, 0.73, -0.14, 0.52, ...] and Vector (Sentence 2) = [0.24, 0.70, -0.12, 0.49, ...]. These vectors are then compared using similarity methods like Cosine Similarity, Dot product. If similarity score is high, then model considers these two sentences are semantically similar.
- Explore the TensorFlow Projector to visualize and understand how similar data points are positioned close together, which will help to illustrate concept of semantic similarity.

5.8 Real World Example

To understand how vector representation, dimensions, and similarity work together in an AI application, go through the following Movie Recommendation System example.

Image: Movie Recommendation System

6. Transformers

6.1 Explanation

Before 2017, language models read text one word at a time, left to right, and tended to forget the start of a long sentence by the time they reached the end. The Transformer changed that. It looks at all the words at once and, for each word, works out which other words matter most for understanding it. That mechanism is called attention.

Take the sentence "The animal didn't cross the street because it was tired." To understand "it", you need to connect it to "animal", not "street". A Transformer does exactly that: when it processes "it", it pays strong attention to "animal". Because attention is calculated for every word in parallel, Transformers also train much faster on GPUs, which is what made today's huge models possible.

6.2 How a transformer processes text

1. **Tokenize the text into pieces called tokens (whole words or parts of words).**
2. **Embed each piece as a vector of numbers.**
3. **Add position information, because attention on its own has no sense of word order.**
4. **Run it through a stack of transformer blocks, where each block is attention followed by a feed-forward network.**
5. **Predict the output, usually the next token.**

6.3 Key terms

Term	What it means
1. Transformer	A neural network architecture based on attention that processes all tokens in parallel; the foundation of modern LLMs.
2. Attention	A mechanism that lets the model weigh how relevant every other piece of the input is when processing one piece.
3. Self-attention	Attention applied within a single sequence, so every token can look at other tokens in the same text (in decoder models, only the tokens before it).
4. Multi-head attention	Running several attention computations in parallel so the model can capture different kinds of relationships at once.
5. Encoder / Decoder	Encoders build a representation of the input (BERT); decoders generate output token by token (GPT, Claude, Llama).

6.4 Architecture diagram

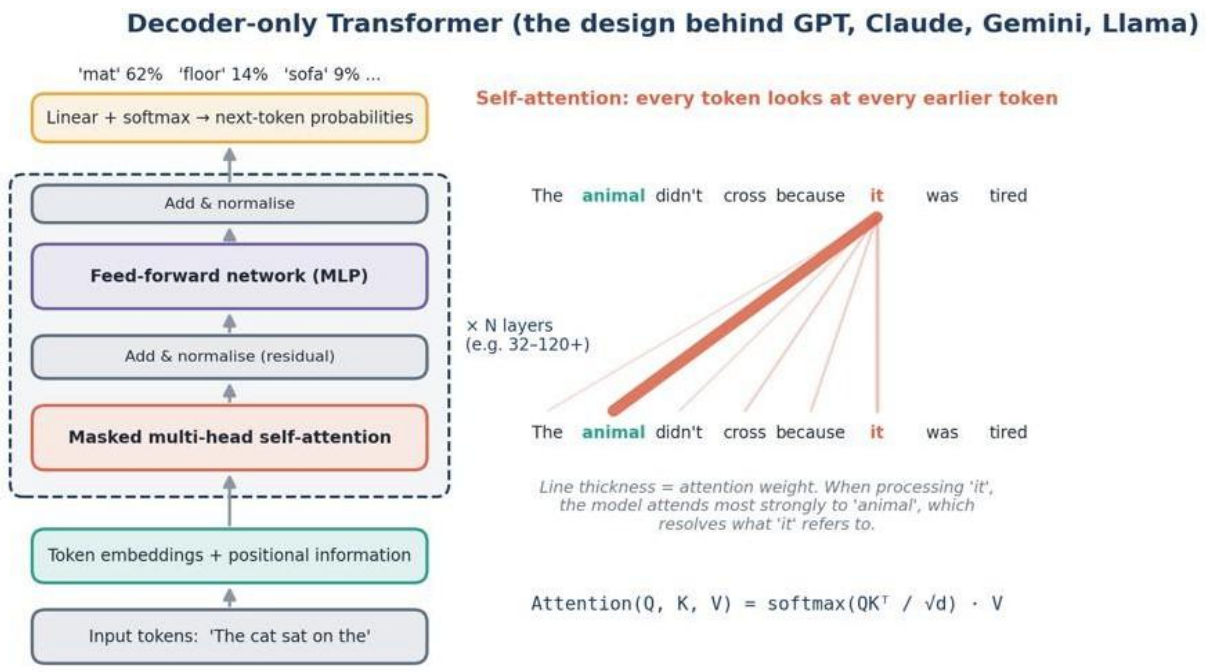


Figure 6: A decoder-only transformer (left) and the attention pattern for the word "it" (right).

6.5 How attention works: Query, Key and Value

Each token's vector is multiplied by three learned weight matrices to produce three new vectors:

- Query (Q): what this token is looking for
- Key (K): what this token offers to others
- Value (V): the information this token passes along if it's selected

In matrix form, with X holding every token's embedding: $Q = X \cdot W_Q$, $K = X \cdot W_K$, $V = X \cdot W_V$. The three W matrices start out random and are learned during training.

Then, for every token:

1. Compare its query against every token's key using a dot product. A high score means "relevant to me."
2. Divide each score by $\sqrt{d_k}$, the square root of the key size. Without this, large scores push softmax to extreme 0/1 values and training become unstable.
3. Apply softmax so the scores become weights that sum to 1.
4. Multiply each token's value vector by its weight and add them up. The result is the token's new, context-aware vector.

The whole calculation as one formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \cdot V$$

6.6 Multi-head attention and masking

- **Multiple heads:** the model runs several attention calculations side by side, each with its own W_Q, W_K and W_V. GPT-2 small, for example, has 12 heads per layer, each working on 64 of the 768 numbers in a token's vector. Nobody assigns the heads jobs; each learns its own pattern during training (for example, looking at the previous word, or linking a pronoun to its noun). Their outputs are joined back together and mixed by one more matrix.
- **Causal mask:** in models that generate text, a token must not see the tokens after it. Scores for future positions are set to $-\infty$ before softmax, which makes their weights zero.

6.7 Inside each transformer block

- **Feed-forward network (MLP):** a small two-layer network applied to each token on its own. Attention is where tokens share information; the feed-forward network is where each token processes what it gathered.
- **Residual connections:** each sub-layer's output is added to its input instead of replacing it, which lets very deep stacks train.
- **Layer normalization** keeps the numbers in a stable range. Most modern models normalize before each sub-layer rather than after it as drawn in Figure 6.

6.8 Training and generating text

- **Pretraining:** the model reads huge amounts of text and learns to predict the next token. No labelling is needed, because the text supplies its own answers.

- **Post-training:** fine-tuning on example conversations and on human or AI feedback turns the raw model into an assistant that follows instructions.
- **Generation:** predict a probability for every possible next token, pick one, add it to the text and repeat. Temperature, top-k and top-p control how random the pick is.

6.9 Example

- For visualization, check out the Transformer Explainer.
- For the working implementation, check out the Transformer Attention Walkthrough notebook.

7. Large Language Models

7.1 What it is?

- LLMs are machine learning models trained on large amount of data such as books, articles, websites, and code repositories to learn pattern in human language.
- They are called “Large” because these types of models are normally made up of millions or billions of parameters that defines the model’s behaviour.
- They learn patterns such as grammar, word relations and conversational context from text and that will train them to answer questions, write code or content, translate languages and many more.
- It operates entirely as a super-charged, next-token (word) prediction engine i.e. Glorified Autocomplete. It predicts the next token based on the tokens that come before it.
- When we provide a prompt, the LLM converts the text into tokens, then assign Token IDs to each and convert those IDs into vector embeddings using Embedding model. And generate an output by calculating probability of next token (word) and repeat this process until the message is finished (Lots of math happens behind this).
- LLMs are autoregressive i.e. an output from one pass becomes an input for the next pass. And this loop continues until the model predicts the complete output.

7.2 How LLMs work

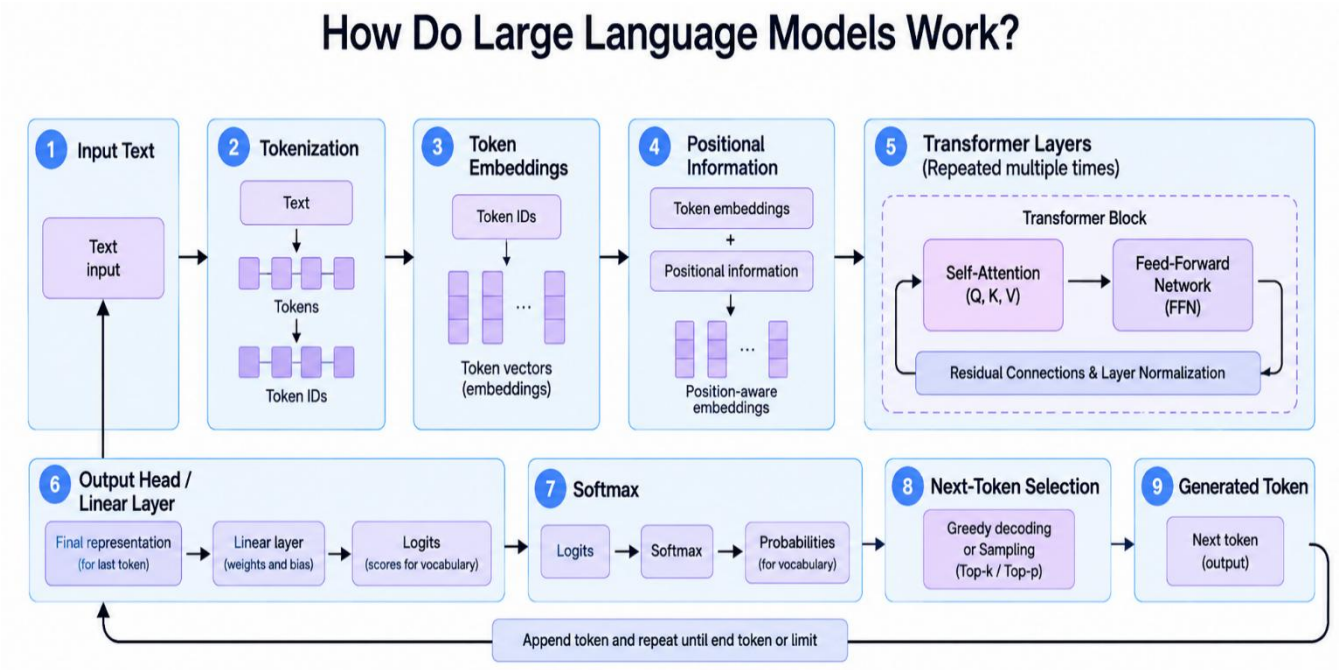


Figure 7: How LLMs work

Tokenization

- **The input text is divided into smaller units called tokens. Each token is then assigned with unique Token ID.**

Embeddings

- **Each Token ID is converted into numerical vector called an embedding, which represents learned features of the token, which are later used by Transformer to understand its meaning in the context.**

Transformer Architecture

- **LLMs are built on type of neural network architecture called "Transformer Architecture" which handles sequence of words and captures patterns in data.**
- **The embeddings are pass through multiple Transformer blocks which consist of many of neural network layers, and at each layer the embeddings are slightly adjusted**

Positional Encoding

- **Transformers also add positional encodings before self-attention or during self-attention, which give information about the position of tokens in a sequence.**
- **The order of words matters in understanding the semantic meaning.**
- **For example, "Dog chased the Cat" and "Cat chased the Dog" have entirely different meaning despite of having the same tokens (or words). So, we use positional encoding to retain information of the word order, enabling parallelization and efficient model training.**
- **The token embedding and positional encoding are added together to create representation that contains tokens information and its position in the sentence.**

Self-Attention

- **Transformer architecture is useful because of its "Self-Attention Mechanism", which allows the model to pay attention to different tokens at different moments.**
- **It helps the model to understand relationships between tokens by determining which tokens are important to each other.**
- **To compute attention, each embedding is projected into three distinct vectors using learned weight matrices (matrix of numerical values that an AI model adjusts during training): Query, Key, Value.**
  - **Query: What information the current token is looking for?**
  - **Key: What information used to determine how relevant a token is to current token?**
  - **Value: What useful information that can be passed from token if it receives attention?**
- **Then model calculates the attention score by comparing Query and Key vectors. The scores are then passed through SoftMax function to obtain attention weights. These weights determine how much each Value vector contributes to the final output.**

Feed-Forward Network

- **After identifying relationships between tokens, FFN further processes the information for each token.**
- **It consists of neural network layers that transforms the token representations and help the model to learn complex patterns.**
- **The output is passed to next Transformer Block, where the self-attention and FFN processes the updated token representations to learn more complex patterns and relationships.**

Output Layer and Logits

- **After passing through Transformer Blocks, the final representations are passed through a linear output layer. This layer does mathematical calculations to assign score i.e. logits for every possible next token.**

Probability and Next-Token Selection

- **The model applies SoftMax function to convert the logits into probabilities for every possible next token.**
- **Based on these probabilities and decoding strategy (like Top-P and Top-k), the model selects the next token.**
- **For example, after the input "The sky is", the model might assign a high probability to the token "blue" and selects it.**

Autoregressive Generation

- **The selected token is added to the existing sequence and becomes the part of the input for the next token prediction.**
- **The model repeats the process of predicting next token and selecting the next token until it generates an end-of-sequence (EOS) or reaches its predefined limit.**
- **This process is called autoregressive generation, where each new token is generated using the tokens that came before it.**

8. LLM Inference

8.1 What are LLMs?

LLMs are Next-Token (Word) Predictors

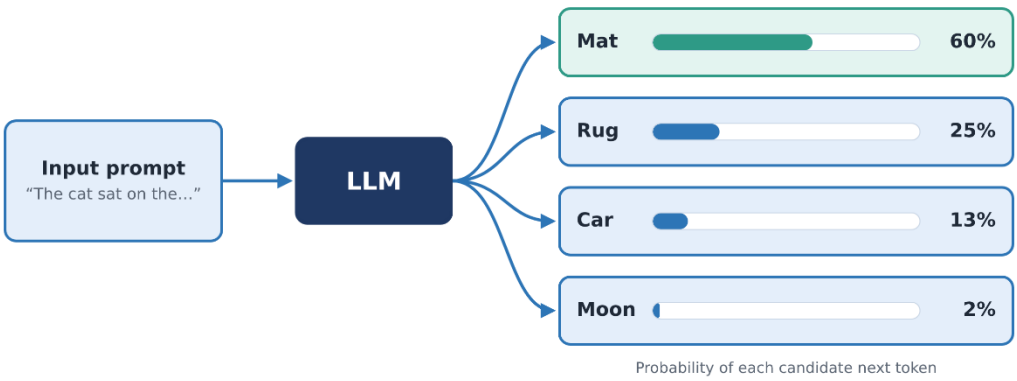


Figure 8: An LLM scores the possible next tokens for the prompt

An LLM generates text by predicting the next token (word) based on the tokens that have already been provided. In the diagram, the input prompt is: "The cat sat on the…"

The LLM processes this input and calculates scores for many possible next tokens based on its learning dataset. These scores are converted into probabilities, representing how likely each token is to be selected as the next token.

For example, the model may assign:

- Mat → 60% probability
- Rug → 25% probability
- Car → 13% probability
- Moon → 2% probability

The token with the highest probability is "Mat", so the next token based on the highest probability can be selected "Mat" as the next token.

8.2 Training vs Inference

Imagine we want to create an LLM.

What is Training?

Training means teaching a model using a large amount of data. We give the dataset to the model, and the model learns patterns from that data. Based on what it learns, the model knows how to respond to different inputs.

The model after this learning process is called a trained model.

What is LLM Inference?

LLM inference is the process of using a trained model to produce an output from new input.

8.3 How Does an LLM Predict the Next Token?

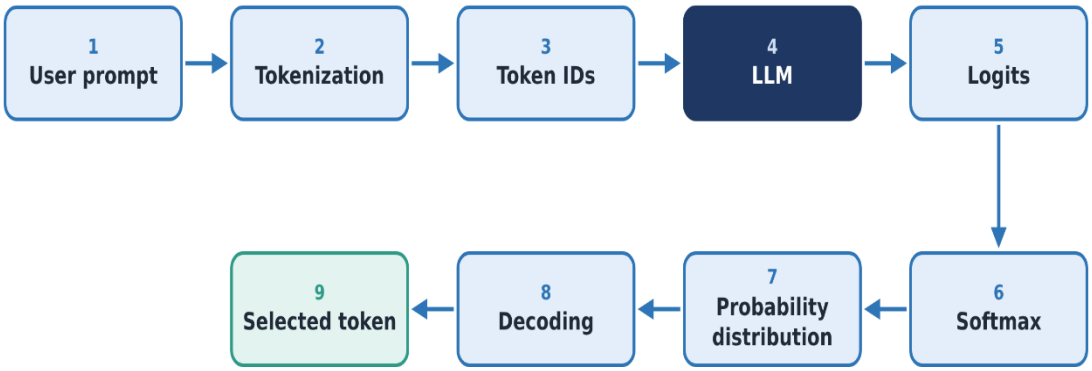


Figure 9: The steps an LLM follows to predict the next token

Tokenization

Before an LLM can process text, the input text is converted into smaller pieces called tokens. These tokens can be complete words, parts of words, punctuation marks, or other text fragments.

Example: Input: "The cat sat on the" → "The", " cat", " sat", " on", " the"

Then these tokens are converted into token IDs. These IDs are based on the tokenizer's vocabulary.

Token	Token ID
“The”	183
“ cat”	742
“ sat”	981
“ on”	192
“ the”	183

Logits

After the input text is converted into token IDs, these tokens are further sent to the LLM. The model produces a numerical score for every possible next token. These raw scores are called logits. Logits indicate how strongly the model matches the token to the previous token. However, logits are not probabilities. They can be positive or negative and do not have to add up to any particular value.

Example: Suppose the input is: “The capital of France is”. The model may produce the following logits:

Token	Logit
Paris	8
London	3
Berlin	2
Mumbai	1

Here, Paris has the highest logit, meaning the model gives it the highest raw score among these candidates.

However, we cannot say that Paris has an 8% or 80% probability just because its logit is 8.

The logits need to be converted into probabilities before we can consider them as probabilities.

Softmax — Converting Logits into Probabilities

Softmax is a mathematical function used to convert the raw logits produced by an LLM into a probability distribution.

Formula

$$P(i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Where:

- P(i) = probability of a token
- z_i = logit of that token
- e = Euler’s number, approximately 2.718
- $\sum_j e^{z_j}$ = sum of the exponentials of all logits

Example: Converting Logits into Probabilities

Input: “The capital of France is”

Step 1: Calculate e^{logit} for each token

Token	Logit	e^{logit}
Paris	8	$e^8 = 2980.958$
London	3	$e^3 = 20.086$
Berlin	2	$e^2 = 7.389$
Mumbai	1	$e^1 = 2.718$

Step 2: Calculate the denominator

Add all the exponential values:

$$2980.958 + 20.086 + 7.389 + 2.718 = 3011.151$$

Step 3: Calculate probability of each token

For Paris:

$$P(\text{Paris}) = \frac{2980.958}{3011.151} \approx 0.98998$$

Similarly:

Token	Probability
Paris	98.998%
London	0.667%
Berlin	0.245%
Mumbai	0.090%

> **Note: Softmax does not select the next token. It only converts the logits into probabilities.**

Decoding

Decoding is the process of selecting the next token from the probability generated by the LLM.

1. Greedy Decoding

Greedy decoding selects the token with the highest probability. For example:

Token	Probability	Result
Paris	98.998%	Selected
London	0.667%	
Berlin	0.245%	
Mumbai	0.090%	

2. Sampling

In sampling, the model selects a token based on the probability distribution rather than always selecting the highest-probability token. This allows lower-probability tokens to sometimes be selected, which can produce more varied responses.

But what decides that a token of lower probability can also be selected?

2.1 Temperature

- Temperature is a decoding parameter that controls how random or predictable the token selection is during LLM inference.
- Temperature is applied to the logits before they are converted into probabilities using Softmax.
- A lower temperature makes the probability centric to highest-probability tokens, making the output more predictable. A higher temperature increases the chance of selecting lower-probability tokens.

Formula

$$P(i) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

Where:

- z_i = logit of the token
- T = temperature
- P(*i*) = probability

Example: Effect of Temperature

Use the logits you already have:

Token	Logit
Paris	8
London	3
Berlin	2
Mumbai	1

a. Temperature = 1

When T = 1, the logits remain unchanged, so the probabilities are the same as in the Softmax example:

Token	Logit	Probability
Paris	8	98.998%
London	3	0.667%
Berlin	2	0.245%
Mumbai	1	0.090%

So, Paris is overwhelmingly likely to be selected.

b. Temperature = 0.5

Now T = 0.5. We divide every logit by 0.5, for example:

$$\text{Paris: } \frac{8}{0.5} = 16$$

Token	New logit (÷ 0.5)	Probability
Paris	16	99.995%
London	6	0.0045%
Berlin	4	0.0006%
Mumbai	2	0.00008%

The probability of Paris became even higher.

> Lower temperature → Sharper distribution → More predictable output

c. Temperature = 2

Divide every logit by 2 to get the new logits, then apply Softmax:

Token	New logit (÷ 2)	Probability after Softmax
Paris	4	86.05%
London	1.5	7.06%
Berlin	1	4.28%
Mumbai	0.5	2.60%

Now Paris is still the most likely token, but the other tokens have a much higher probability than before.

2.2 Top-K Sampling

Top-K sampling is a decoding technique that limits the number of candidate tokens considered for the next-token selection.

The value K represents the number of highest-probability tokens that will remain as candidates.

For example, suppose the LLM produces this probability distribution:

Token	Probability
Paris	40%
London	25%
Berlin	15%
Rome	10%
Tokyo	5%
Delhi	3%

The total probability is: 40 + 25 + 15 + 10 + 5 + 3 + 2 = 100%

Suppose K = 3: the model keeps only the 3 highest-probability tokens: Paris → 40%, London → 25%, Berlin → 15%.

The remaining tokens are removed from the candidate set: Rome, Tokyo, Delhi.

What happens next?

The remaining probabilities are renormalized so that they add up to 100%. The remaining probability mass is: 40 + 25 + 15 = 80%

Now calculate the new probabilities:

$$P(\text{Paris}) = \frac{40}{80} = 50\%$$

$$P(\text{London}) = \frac{25}{80} = 31.25\%$$

$$P(\text{Berlin}) = \frac{15}{80} = 18.75\%$$

So, Top-K is used to reduce the candidates to the tokens that have the highest probability.

2.3 Top-P / Nucleus Sampling

Top-P sampling selects the most likely tokens until their total probability reaches the given P value.

Example: Use the same distribution:

Token	Probability
Paris	40%
London	25%
Berlin	15%
Rome	10%
Tokyo	5%

Suppose Top-P = 80%. Now add probabilities from highest to lowest:

- **Paris: 40% (cumulative: 40%)**
- **Add London: 40 + 25 = 65%**
- **Add Berlin: 40 + 25 + 15 = 80%**

8.4 Final Token Selection

After applying Top-K or Top-P, the model has a smaller list of tokens to choose from. The model does not always select the first or highest-probability token. It chooses a token based on the probabilities. The model's learning also affects these probabilities.

Temperature controls how predictable or random the selection is:

- **Lower temperature → more predictable output.**
- **Higher temperature → more random and varied output.**

8.5 Practical Demonstration

Experiment Setup

The experiment was implemented in a Jupyter Notebook using Python and the OpenRouter API.

Tools used:

- **Python**
- **Jupyter Notebook**

- **OpenRouter API**
- **Pandas**

The notebook contains the complete implementation, different parameter values, generated outputs, and observations.

GitHub: You can check the link for the implementation: LLM Inference

9. Foundational Models

9.1 What it is

- **Foundational models are the AI models which are trained on large amount of data to do a wide range of downstream tasks (the task that we want to solve), such as sentiment analysis of text, question answering, text summarization, image generation etc.**
- **They can work with text, images, audio, video, code and other type of data.**
- **Imagin building an AI assistant, we can build a separate model for answering questions, another for summarizing documents, and another for generating text. That could be a lot of unnecessary work. Instead of that, we can start with a model trained on broad range of data and adapt it for different tasks.**

9.2 Working

Pre-training

- **The model is trained using self-supervised learning technique (SSL).**
- **In SSL, model is trained using data that does not have any labels or answers provided. Instead of needing people to label data, the model finds the patterns and creates its own labels from data automatically i.e. model derives targets from unlabelled data.**
- **SSL doesn't mean there is no human intervention, it means that humans does not need to manually label every training example with answer (target).**

- The model trains itself by playing endless guessing games. It hides the piece of data from itself and tries to predict it.
- For example, consider the sentence “The sky is blue.” During training, the training process may hide the word “blue” and ask the model to predict the missing word from the surrounding context. Suppose in first iteration model predicts the word “green” (which is a wrong), training process calculates the loss and backpropagates then adjust the parameters and predicts again. This process continues till model improves its predictions.
- This allows the model to learn useful information by teaching itself from the data.
- For text, it hides the word and guess it based on the context.
  For image, it hides the patch of an image and try to reconstruct what is missing.

Adaptation

- A pre-trained model can be adapted to specific requirement.
- This is done by Fine-Tuning.
- Fine tuning is the process of continuously training a foundational model on targeted, domain-specific dataset to permanently alter its behaviour and knowledge.
- Example includes fine tuning a foundational model for medical field, finance, customer support etc.
- The benefit of fine tuning is that we don’t have to train a model from scratch, we take an already trained model which can understand human language.

Deployment

- The adapted i.e. fine-tuned model is integrated into an application such as a chatbot, image-generation system, coding assistant, customer support, or other AI-powered application.

9.3 Types of FMs

Large Language Models (LLMs)

- These models are primarily trained on text and can be adapted for tasks such as text generation, translation, summarization, chatbot interactions, and question answering etc.
- Example: BERT, GPT, Claude, and Llama.

Vision Foundation Models

- These models are trained on image datasets to perform tasks like image generation, object detection, and image classification.
- These models can be fine-tuned for specific applications, such as medical image analysis or object recognition in autonomous vehicles.
- Another capability is the generation of an images from text.
- Example: DALL-E(OpenAI), Stable diffusion (Stability AI), Imagen (Google).

Code Generation Models

- These models can assist with completing, debugging, and generating code in different programming languages.
- Example: OpenAI Codex, Code Llama (Meta), StarCoder (BigCode), CodeGemma (Google).

Multimodal Foundational Models

- These models are trained to process diverse data types, such as text, images, audio, or video.
- Example: Gemini (Google), GPT-6 Astra (Open AI), Claude (Anthropic), Llama 4 (Meta).

9.4 How To Pick Right AI Foundation Model

Step 1: Articulate Use case

- First, clearly define the problem organization want to solve using AI.
- This will help to narrow down the suitable models.
- For example, an organization may need an AI model to summarize documents, generate code, answer customer queries etc.

Step 2: List Model Options

- Identify the available foundation models that could perform required tasks.
- The selection should focus on model's relevancy to the use case.

Step 3: Identify Model Characteristics

- Compare selected models based on their characteristics, including:
  - Model Size: Number of parameters in the model
  - Performance: Ability to complete the required task accurately
  - Cost: API usage and Token charges, computation resources maintenance expenses
  - Risks: Potential bias, hallucinations, security concerns, data privacy leaks.

Step 4: Refine Selection

- Refine the shortlist by checking how well each model's capabilities align with specific requirements of the use case.
- For example, if an organization wants to build a code generation application, it should prioritize models with strong coding capabilities rather than selecting general purpose models.

Step 5: Testing

- Test the shortlisted Foundational Model using representative inputs to evaluate their performance against:
  - Accuracy: How correctly the model performs the required tasks
  - Reliability: How consistently the model produces the appropriate results
  - Speed/Latency: How quickly or slowly the model responds
- Compare the test results to identify each model's strengths and weaknesses.

Step 6: Choose the best value

- Select the Foundational Model that offers best balance of accuracy, reliability, speed, cost and security.
- The largest model is not always the best choice as smaller models may meet the requirements at lower cost.

10. Multimodal AI

Multimodal AI is AI that can understand and work with more than one type of input, such as text, images, audio, or video.

These different types of information are called modalities.

A multimodal AI system can use information from multiple modalities together to understand a user's request and generate a relevant response.

10.1 Modalities

A modality refers to a type or format of data that an AI model can process or generate. Multimodal AI models can work with multiple types of data together.

- **Text: Written information, such as questions, instructions, and documents.**
- **Image: Visual information, such as photographs, charts, and diagrams.**
- **Audio: Sound information, such as speech, music, and recordings.**
- **Video: A sequence of visual frames over time, sometimes accompanied by audio.**

Example: A multimodal AI model can analyse a picture, understand a question about it, and generate a text response.

> **Note: The modalities supported depend on the model and its capabilities.**

10.2 How Multimodal AI Works

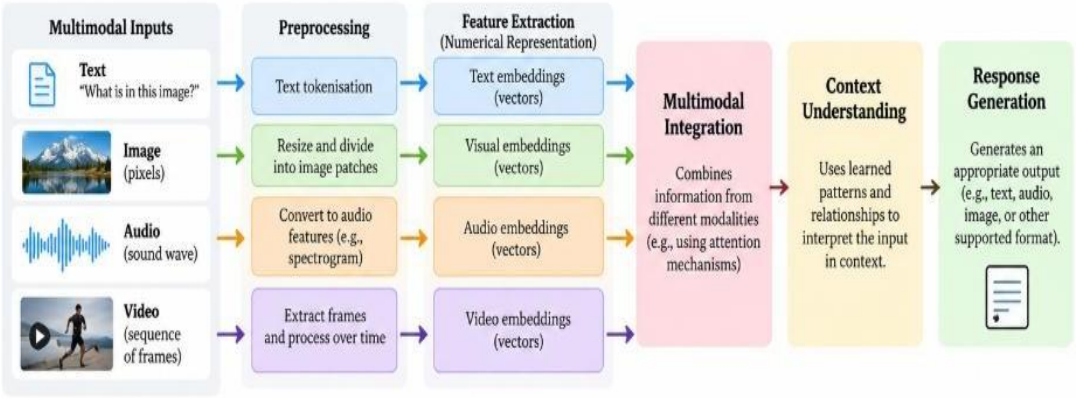


Figure 14: How Multimodal AI Works

The exact internal architecture can be different for different models.

Some systems use separate components to process different modalities before combining their representations. Other systems are designed to handle multiple modalities more directly.

The main idea is that information from different sources can be used together to understand the user's request.

10.3 Cross-modal Understanding

Cross-modal understanding means using information from different modalities together. For example, a model can use:

- **An image to understand visual information**
- **Text to understand the user's question**

  and combine both to generate an answer.

This allows the AI system to connect different forms of information rather than treating every input type separately.

10.4 How Multimodal Models Process Text, Images, Audio, and Video

Multimodal AI models can process different types of data and combine relevant information to generate meaningful responses. The processing method depends on the input type and the model's architecture.

Step 1: Text Processing

The model divides text into smaller units called tokens and converts them into numerical representations called embeddings. These representations help the model process language and understand the user's instructions.

Example: A user asks, "Describe what is happening in this video."

Step 2: Image Processing

The model processes images using a vision encoder. Images divided into smaller regions called patches, which are converted into numerical representations. These representations capture visual features such as objects, shapes, colours, and spatial relationships.

Example: The model identifies a person, a car, and a road in an image.

Step 3: Audio Processing

The model processes audio using techniques designed for sound signals. Depending on the model, audio may be converted into numerical features or processed directly by an audio encoder. These representations can help the model recognise speech, identify sounds, and interpret spoken information.

Example: The model processes a recording and identifies the words spoken by a person.

Step 4: Video Processing

Video contains a sequence of frames displayed over time. The model may extract selected frames and process their visual information to identify objects, actions, and changes between frames. If the model supports audio input, it may also analyse the video's audio track.

Example: The model analyses a video and identifies that a person enters a room, sits on a chair, and starts speaking.

Step 5: Combining Different Modalities

After processing the available inputs, the model connects relevant information using techniques such as attention mechanisms or other multimodal integration methods. This helps it understand relationships between text, images, audio, and video.

Example: A user provides a video with audio and asks, "What is the person doing, and what are they saying?" A capable multimodal model can use the visual information to describe the person's actions and the audio information to identify their spoken words.

Step 6: Response Generation

The model uses the processed information and the user's instructions to generate an appropriate output. Depending on its capabilities, the output may be text, audio, images, or another supported format.

Example: After analysing a video, the model generates a text summary describing the actions and dialogue it could identify.

> **Note: Not every multimodal model supports all four input types. The exact processing techniques and supported outputs depend on the model's architecture and API.**

10.5 Multimodal AI and Hallucination

Multimodal AI can also produce incorrect or unsupported information. For example, a model may:

- **Incorrectly identify an object.**
- **Misinterpret a diagram.s**
- **Misread text in an image.**
- **Describe something that is not actually present.**
- **Make an unsupported statement about unclear visual information.**

10.6 Practical Implementation – Multimodal AI

A practical implementation of Multimodal AI is available in the Jupyter Notebook linked below. The notebook demonstrates how a multimodal model processes an image along with a text prompt and generates a response. You can access the project from this link: Multimodal AI

11. Open vs Closed AI Models

AI models can be made available in different ways. Some models provide access to their trained weights, while others are available mainly through a provider's application or API.

These differences affect how developers' access, customize, deploy, and maintain AI models. They can also influence infrastructure requirements, performance, and cost.

11.1 Understanding Open-Weight and Open-Source AI Models

1. Open-Weight AI Models

An open-weight AI model is an AI model whose trained weights are available for people to access and use, depending on its license.

What are weights?

Weights are numbers that an AI model learns during training. These numbers help the model recognize patterns and give answers.

How do weights work?

For example, suppose we give an AI model a picture of a dog. The model uses its existing weights to understand the picture and predict that it contains a dog.

We do not need to give the weights separately every time we provide an image. The model already has its trained weights.

Can we improve the model?

Yes. If the model makes mistakes while identifying dogs, we can train it further using correctly labeled dog images. This process is called *fine-tuning*. It updates the model's weights and may help it make better predictions.

Example

Some Qwen models allow developers to download their trained weights and run them on their own computers or servers, provided they meet the model's requirements and license terms.

> **Key Takeaway: Open-weight models allow developers to access their trained weights. They can use these models and, where permitted, fine-tune them for specific tasks.**

2. Open-Source AI Models

An open-source AI model is a model that allows people to use, study, modify, and share its available components according to its license.

How does it work?

Developers can access the model's available code and other components to understand how it works. Depending on the model and its license, they may also modify the code or share their changes with others.

Example

The BLOOM language model, developed through the BigScience research collaboration, is an example of an openly released language model. Its publicly available resources allow researchers and developers to explore and use the model under its applicable terms.

> **Key Takeaway: Open-source AI models give developers the freedom to study, use, and modify the available components under the applicable license. However, not every model provides all its training data or other components.**

11.2 Closed/Proprietary AI Models

A closed or proprietary AI model is an AI model whose trained weights are not publicly available for people to download and use freely. These models are generally accessed through a company's website or API.

The company that provides the model manages its underlying technology and controls how it can be accessed and used.

How do closed models work?

Users send their questions or instructions to the model through a website or API. The model processes the input and returns a response. Users do not need to download or manage the model's underlying weights.

Real-World Example

ChatGPT is an example of a service that provides access to OpenAI's proprietary GPT models. Users can ask questions and receive answers through the ChatGPT interface without downloading the model's weights.

Similarly, Anthropic provides access to its proprietary Claude models through its services and APIs.

> **Key Takeaway: Closed/proprietary models allow users to access AI capabilities through provider-managed services, but their underlying trained weights are generally not publicly available.**

11.3 Model Access

Model access refers to how users connect to and use an AI model.

Open-Weight Models: These models can often be downloaded and run on a local computer or server, depending on their license and hardware requirements. They can also be accessed through APIs or third-party platforms.

Closed/Proprietary Models: These models are generally accessed through the provider's website or API. The provider manages the underlying infrastructure when offering a hosted service.

Example: A developer can run a suitable open-weight model locally or connect an application to OpenAI's API to generate responses.

11.4 Customization and Fine-Tuning

Customization means adapting an AI model to meet specific requirements.

Prompting: Providing instructions to guide the model's responses without changing its trained weights.

Fine-Tuning: Training an existing model further using specific examples to improve its performance on a particular task. This process updates some or all of its weights.

Open-weight models may support fine-tuning using suitable tools and hardware, subject to their licenses. Closed models may also support fine-tuning if the provider offers that capability.

Example: A company can fine-tune a suitable AI model using customer-support examples to help it answer common customer questions.

11.5 API-Based Deployment vs Self-Hosting

Deployment means making an AI model available for people or applications to use. Hosting means where the AI model runs.

1. Open-Weight Models

Open-weight models can often be downloaded and run on your own computer or server, depending on the model's license and your computer's capabilities. They can also be used through APIs.

Example: A developer downloads a suitable Qwen model and runs it on their own computer.

2. Closed/Proprietary Models

Closed models are generally used through the company's website or API. The company manages the servers that run the model when providing a hosted service. Users usually cannot download the model's weights.

Example: A developer uses OpenAI's API to add AI responses to an application without running the model on their own server.

12. AI Hallucinations

An AI hallucination occurs when an AI model generates information that is incorrect, unsupported, fabricated, or inconsistent with the information available to the model.

The generated response may appear fluent, confident, and factually plausible, even though the information is not supported by reliable evidence.

Example

Prompt / response	Text
User	Who developed the Java programming language?
Correct response	Java was originally developed by James Gosling and his team at Sun Microsystems.
Hallucinated response	Java was developed by Dennis Ritchie at Bell Labs in 1983.

The second response is grammatically correct and sounds plausible because Dennis Ritchie is associated with programming languages—but the claim is incorrect.

This is an example of factual hallucination.

12.1 Why Do Hallucinations Occur?

A LLM does not operate like a traditional database. A simple view is:

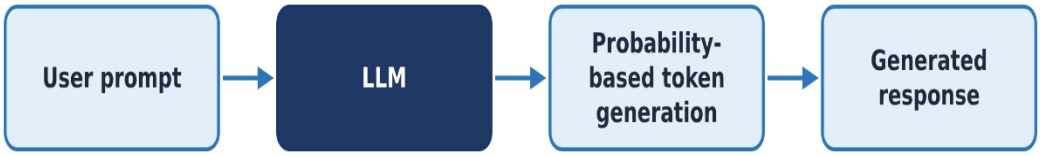


Figure 10: A simple view of how an LLM produces a response

The model predicts what text is expected to come next in the input based on patterns learned during training.

Therefore, fluent language does not always mean the information is correct.

A model can give a convincing answer even when it does not have enough evidence to support the answer.

12.2 Hallucination vs. Normal Error

Not every incorrect response should necessarily be classified in exactly the same way.

1. **Model gives an incorrect calculation: usually reasoning or arithmetic error.**
2. **Model invents a non-existent API.**
3. **Model invents a research paper.**
4. **Model gives an outdated fact.**
5. **Model contradicts provided documentation.**

The key characteristic is that the model presents unsupported or fabricated information as though it were valid.

12.3 Types of Hallucinations

1. **Factual Hallucination — The model provides information that is factually incorrect or not true.**
2. **Fabrication — The model invents information, such as people, research papers, sources, or events that do not exist.**
3. **Contextual Hallucination — The model provides an answer that contradicts or ignores the information given in the context.**
4. **Citation Hallucination — The model generates fake, incorrect, or unsupported references and citations.**
5. **Code/API Hallucination — The model generates code, methods, libraries, or APIs that do not actually exist or work as described.**
6. **Multimodal Hallucination — The model incorrectly interprets information from images, audio, or video.**
7. **Temporal Hallucination — The model provides outdated information as if it were current or valid.**
8. **Instruction-Based Hallucination — The model introduces unsupported information or fails to follow the given instructions or constraints.**

12.4 Hallucination Documentation Record

Example 1 — T&F Roaster

Type: Contextual Hallucination

Explanation: The model interpreted an ambiguous term using an unrelated meaning instead of the context provided in the document.

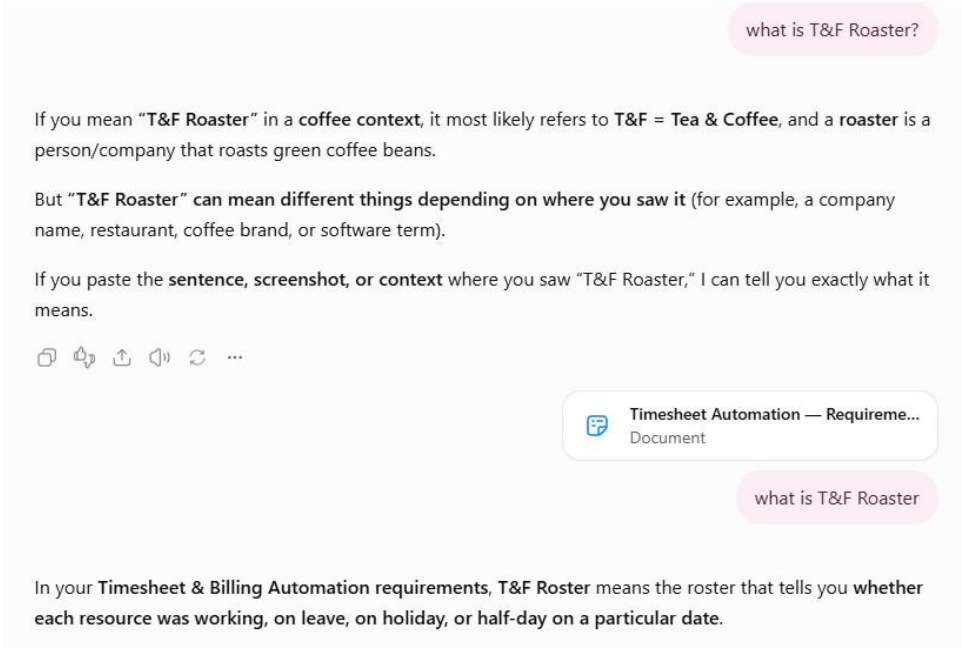


Figure 11: Example 1, T&F Roaster (contextual hallucination)

Example 2 — Dog & Muffin Image

Type: Multimodal Hallucination

Explanation: The model may incorrectly identify objects in an image or confuse visually similar objects.



Figure 12: Example 2, Dog & Muffin image (multimodal hallucination)

Example 3 — JWST

Type: Factual Hallucination

Explanation: The model generated an incorrect or unsupported factual claim about discoveries made by the James Webb Space Telescope.

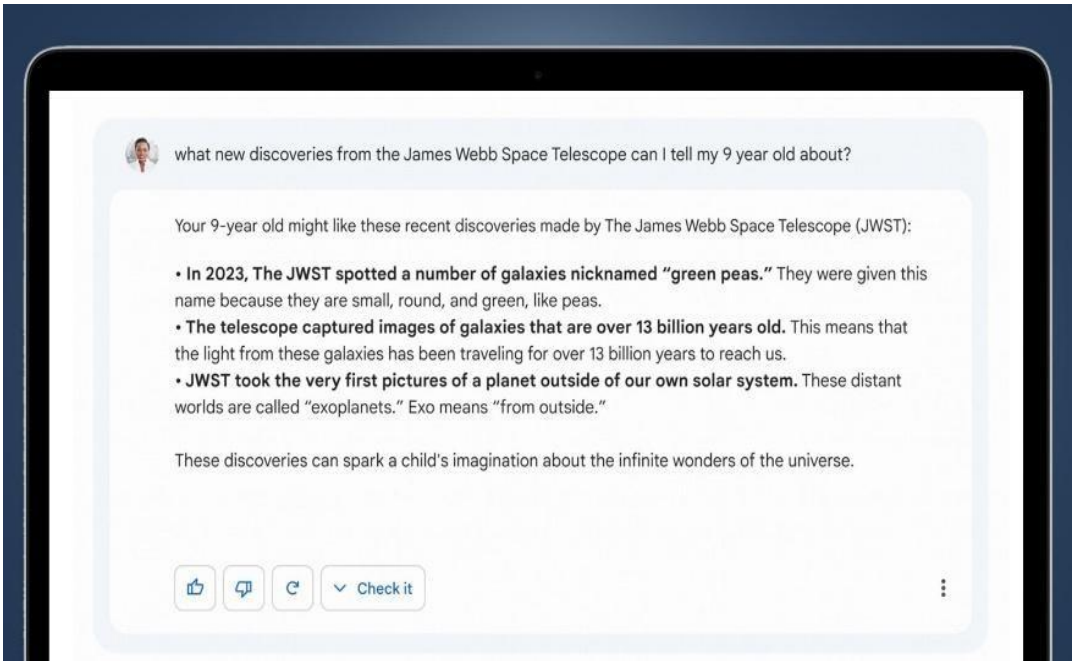


Figure 13: Example 3, JWST (factual hallucination)

12.5 Root Causes of AI Hallucinations

1. **Insufficient Knowledge — The model does not have enough relevant information to provide a reliable answer.**
2. **Outdated Knowledge — The model relies on information that is no longer current or accurate.**
3. **Ambiguous Prompts — The question is unclear or has multiple possible interpretations, causing the model to make assumptions.**
4. **Lack of Context — The model does not have enough context to understand the user's intended meaning.**
5. **Poor Retrieval — In RAG systems, incorrect, incomplete, or irrelevant information is retrieved and used to generate the answer.**
6. **Model Overconfidence — The model generates an answer confidently even when it has insufficient evidence to support it.**
7. **Training Data Issues — Errors, biases, inconsistencies, or unreliable information in training data can influence the model's responses.**
8. **Context Conflicts — Conflicting information within the prompt, retrieved documents, or conversation can cause the model to generate an incorrect response.**
9. **Complex Reasoning — Multi-step reasoning, calculations, or logical problems can cause the model to reach unsupported conclusions.**
10. **Multimodal Misinterpretation — The model may incorrectly interpret information from images, audio, or video.**

12.6 Hallucination Detection

Hallucinations can be detected by checking whether the model's response is correct, supported, and consistent with the available information.

1. **Ground-Truth Comparison — Compare the AI answer with the correct answer.**
2. **Source Verification — Check if the sources given by AI are real and correct.**
3. **Context Verification — Check if the answer matches the information provided.**
4. **Grounding Check — Check if the answer is supported by the available documents.**